

Clases-Contenedoras.pdf



JesusLagares



Programación Concurrente y de Tiempo Real



3º Grado en Ingeniería Informática



**Escuela Superior de Ingeniería
Universidad de Cádiz**



[Accede al documento original](#)

antes



**Descarga sin publi
con 1 coin**



Después

WUOLAH



Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo espacio



Clases Contenedoras



Las **clases contenedoras concurrentes** forman parte de la **API de alto nivel** (`java.util.concurrent`, introducida en Java 5). Nos ofrecen **estructuras de datos seguras para múltiples hilos** sin necesidad de que el programador gestione explícitamente la exclusión mutua o la sincronización.

Explicación

Son versiones concurrentes de las colecciones clásicas de Java:

- listas, colas, mapas, sets, etc.

La forma en la que implementan su protección:

- **no se basa en `synchronized` de manera simple,**
- ni en un único cerrojo centralizado (salvo en las versiones "synchronizedX" de la API antigua),
- sino en técnicas internas más avanzadas como **bloqueos finos, bloqueo por segmentos, estructuras lock-free o wait-free, etc.**

Pero todo eso es **transparente** para el programador.

Cuando utilizas un contenedor concurrente, ya tienes garantizada exclusión mutua y sincronización.

No debes añadir cerrojos externos.

Vamos, que no tienes que preocuparte por nada. Simplemente al usarlas tú ya sabes que te lo garantizan y punto. No hay que saber cómo internamente gestionan la exclusión mutua y la sincronización.

Aunque si quieres saberlo hay que revisar la definición de la API.

Ejemplos

Algunos ejemplos son:

- `ConcurrentHashMap`.
- `ConcurrentSkipListMap`.
- `ConcurrentSkipListSet`.

Colas Sincronizadas

Hacemos especial hincapié en el subconjunto de las colas sincronizadas, ya que en la asignatura se usarán para resolver problemas como el del Productor - Consumidor.

Son *thread-safe* por diseño y se **auto-sincronizan**: un hilo que intenta insertar o extraer puede **bloquearse automáticamente** según la capacidad y estado.

Lo único que debes asegurarte es de cómo realizan esta sincronización, o sus capacidades, ya que si quieres resolver el problema del Productor - Consumidor con buffer infinito no vas a poder usar una `SynchronousQueue` por ejemplo (porque esta cola no tiene capacidad: cada inserción (put) requiere que un consumidor esté esperando exactamente en ese momento para recibir el ítem).

Ejemplos:

`LinkedBlockingQueue`

- Cola FIFO, enlazada, con capacidad opcional.
- Los hilos productores esperan si está llena.
- Los consumidores esperan si está vacía.

`ArrayBlockingQueue`

- Implementada sobre un array circular.
- Capacidad fija.
- Alto rendimiento cuando la capacidad no cambia.

`SynchronousQueue`

- **Capacidad 0.**

- Un productor **no puede insertar** si no hay un consumidor esperando simultáneamente.
- Es un “punto de encuentro” entre hilos.

Perfecta para ejecutores donde las tareas se entregan directamente a un trabajador.

PriorityBlockingQueue

- Cola con prioridad, no FIFO.
- Nunca se llena (no tiene capacidad fija).

DelayQueue

- Los elementos solo pueden extraerse cuando expira el tiempo asociado a cada uno.
- Muy útil para planificadores o reintentos diferidos.

```
BlockingQueue<Integer> cola = new LinkedBlockingQueue<>();

// Productor
new Thread(() -> {
    try {
        while (true) {
            cola.put(1); // espera si la cola está llena
        }
    } catch (InterruptedException e) {}
}).start();

// Consumidor
new Thread(() -> {
    try {
        while (true) {
            int x = cola.take(); // espera si la cola está vacía
        }
    } catch (InterruptedException e) {}
}).start();
```